READ_API_KEY Permission — Variant-Safety Analysis

Status: FIXED (P2-1, 2026-06-14). The permission name is now variant-suffixed via the `${apiKeyPermission}` manifest placeholder in BOTH apps — release keeps the byte-identical `digital.vasic.lava.permission.READ_API_KEY` (existing release grants survive); debug uses `digital.vasic.lava.permission.dev.READ_API_KEY`. A device with both variants installed can no longer hit `INSTALL_FAILED_DUPLICATE_PERMISSION`. Verified by `:app`/`:api-app` `processDebugManifest` + `processReleaseManifest` merges: debug resolves to the `.dev` name (uses-permission + `<permission>` + provider `readPermission`), release resolves byte-identically to the original. Runtime alignment: `ApiKeyProvider.attachInfoForTest` now reads the variant-aware `BuildConfig.API_KEY_PERMISSION` (api-app) instead of the release-only `AppLinkContract.PERMISSION_READ_API_KEY` constant. Changed files: `app/build.gradle.kts`, `api-app/build.gradle.kts`, both `AndroidManifest.xml`, `api-app/.../ApiKeyProvider.kt`. See §1.4 ("Minimal fix") below for the design.

Date: 2026-06-14 **Author:** analysis (read-only, manifest + build-file evidence only) **Scope:** Is the on-device search 401 (key read returned null because `digital.vasic.lava.permission.READ_API_KEY` was not granted) a real production permission-variant bug, or a test-VM artifact of mixed debug/release install history?

---

1. The facts (manifest + build-file cites)

Permission NAME — FIXED literal, NOT variant-suffixed

- **DEFINED** by api-app: `api-app/src/main/AndroidManifest.xml:11-13`

```
<permission
    android:name="digital.vasic.lava.permission.READ_API_KEY"
    android:protectionLevel="signature" />
```

- **GUARDS the provider** (same fixed name): `api-app/src/main/AndroidManifest.xml:93`

android:readPermission="digital.vasic.lava.permission.READ_API_KEY"
- **USED** by the client: app/src/main/AndroidManifest.xml:9 <uses-permission android:name="digital.vasic.lava.permission.READ_API_KEY" />

The permission name carries **no `.dev` suffix and no `${...}` placeholder** in either manifest. It is byte-identical across debug and release of BOTH apps.

protectionLevel — `signature` (the safe variant)

api-app/.../AndroidManifest.xml:13 declares `protectionLevel="signature"` (NOT `signatureOrSystem`, NOT `dangerous`). A signature permission is granted at install time to any app signed with the **same certificate** that signed the app which DEFINED the permission. No runtime grant dialog; the OS decides purely by cert match.

Authority — variant-suffixed (CORRECT)

The provider AUTHORITY, by contrast, IS variant-aware:

- api-app/.../AndroidManifest.xml:91 `android:authorities="${apiKeyAuthority}"`
- Release placeholder: api-app/build.gradle.kts:95 `manifestPlaceholders["apiKeyAuthority"] = "digital.vasic.lava.api.keyprovider"`
- Debug placeholder: api-app/build.gradle.kts:158 `manifestPlaceholders["apiKeyAuthority"] = "digital.vasic.lava.api.dev.keyprovider"`

The client mirrors this with `API_TARGET_PACKAGE` (app/build.gradle.kts:116 release default `digital.vasic.lava.api`; :173 debug override `digital.vasic.lava.api.dev`) — so the client queries the correct authority per variant. The authority dimension is sound and is not the subject of this bug.

applicationIds

App	Release applicationId	Debug applicationId
client (app)	`digital.vasic.lava.client` (build:51)	…`client.dev` (build:169 `applicationIdSuffix = ".dev"`)
api-app	`digital.vasic.lava.api` (build:71)	…`api.dev` (build:150 `applicationIdSuffix = ".dev"`)

Signing — same keystore for both apps, per build type

Both `app/build.gradle.kts:135–148` and `api-app/build.gradle.kts:116–129` use the IDENTICAL signing inputs: debug → `keystores/debug.keystore` (alias `debug`), release → `keystores/release.keystore` (alias `release`), passwords from the same `.env`. `api-app/build.gradle.kts:14–17` states this verbatim: "this module REUSES the EXACT signing block :app uses". Therefore:

- **release client cert == release api-app cert** (release.keystore)
- **debug client cert == debug api-app cert** (debug.keystore)
- **release cert != debug cert** (different keystores)

---

2. Per-install-combination grant verdict

The signature-permission grant rule: the OS grants `READ_API_KEY` to the client at install time **iff** the client's signing cert == the cert of the app that DEFINED the permission (the api-app), AND the permission DEFINITION currently installed on the device belongs to a package signed with that same cert.

(a) release client + release api-app — THE PRODUCTION PAIR — GRANTS ✅

Both signed by `release.keystore`. The release api-app defines `READ_API_KEY` (signature). The release client requests it. Certs match → **granted at install**. `ApiKeyClient.read()` → provider query passes the `readPermission` check → key returned → search authenticates. **SAFE.**

(b) debug client + debug api-app — matched dev pair — GRANTS ✅

Both signed by `debug.keystore`. Definition + request certs match → granted. Authority `digital.vasic.lava.api.dev.keyprovider` matches the debug `API_TARGET_PACKAGE` (…`api.dev`). **SAFE** (when cleanly installed — see §2d).

(c) BOTH release + debug variants co-installed — RISK ⚠️ (test-VM trap)

Because the permission NAME is a FIXED literal (no `.dev` suffix), the release api-app and the debug api-app BOTH try to DEFINE the **same** permission name `digital.vasic.lava.permission.READ_API_KEY`, but with **different signatures** (release.keystore vs debug.keystore). This is the classic Android duplicate- permission trap:

- The **second** api-app to install hits `INSTALL_FAILED_DUPLICATE_PERMISSION` (a permission with that name already exists, owned by a differently-signed package), OR
- If install ordering/PackageManager state lets both coexist, the permission's *owning definition* is whichever package PackageManager attributes it to. A client signed with the OTHER cert than the current definition-owner is then **denied** the grant — exactly the observed `ApiKeyClient.read()` → null → 401.

This is precisely the mixed debug/release install history described in the bug context, and it is the SAME root as the earlier `INSTALL_FAILED_DUPLICATE_PERMISSION`. It manifests ONLY when both differently- signed variants of the api-app are present on one device.

(d) Upgrade path: release api-app installed, then updated — GRANTS ✅

An update replaces the same package (`digital.vasic.lava.api`) signed by the same release cert. The permission definition is re-declared by the same owner with the same signature; no re-definition conflict. The already-installed release client's grant persists. **SAFE.**

---

3. VERDICT

Production (matched release pair, clean install) is SAFE. A user who installs ONLY the release client + ONLY the release api-app — both signed by `release.keystore`, the only combination shipped to end users via the Play Store / Firebase release channel — gets `READ_API_KEY` granted at install (§2a), so `ApiKeyClient.read()` returns the key and search authenticates. The fixed permission name is not a problem when only ONE signing identity defines it on the device.

The observed 401 is a test-VM artifact, not a production bug. It is caused by having **both** differently-signed api-app variants (debug `…api.dev` from debug.keystore AND release `…api` from release.keystore) in the install

history of one VM (§2c). Two differently-signed packages contending to define the same fixed-name permission is what broke the grant — the same condition that earlier produced `INSTALL_FAILED_DUPLICATE_PERMISSION`. Real users do not co-install a debug and a release variant of the api-app.

Is there a latent risk worth hardening?

Yes — a **defense-in-depth** one, not a production-breaking one. Today the permission name is a fixed literal while every other cross-app identifier (applicationId, provider authority, `API_TARGET_PACKAGE`) is correctly variant-suffixed. The fixed name is the ONE cross-app identifier that can collide between variants. It bites:

- developers/CI/QA running mixed debug+release installs (the present case), and
- theoretically any user who somehow ends up with both variants.

Minimal fix (recommended — eliminates the collision class entirely)

Variant-suffix the permission name so debug and release never define the same name, exactly as the authority is already suffixed:

1. api-app: make the permission name a manifest placeholder, e.g. `android:name="${apiKeyPermission}"` on BOTH the `<permission>` (line 11-13) and the provider's `android:readPermission` (line 93), with `manifestPlaceholders["apiKeyPermission"]` set to `digital.vasic.lava.permission.READ_API_KEY` (release, build:~95) and `digital.vasic.lava.permission.dev.READ_API_KEY` (debug, build:~158).
2. client (`app`): mirror the same placeholder on the `<uses-permission>` (line 9), with the matching per-variant value (release default + debug override under build:166-175), plus a `BuildConfig` field if any runtime code references the permission name (§6.R: no source literal — drive from the placeholder/BuildConfig, same pattern as `apiKeyAuthority` / `API_TARGET_PACKAGE`).

After the fix, debug defines `…dev.READ_API_KEY` (debug cert) and release defines `…READ_API_KEY` (release cert); the two never collide, `INSTALL_FAILED_DUPLICATE_ PERMISSION` cannot recur, and a mixed-install VM grants each variant its own permission. This makes the test surface match production behavior and removes the last fixed-literal cross-app identifier.

Priority: LOW for production correctness (production is already safe per §2a/d); MEDIUM for developer/QA hygiene + anti-bluff fidelity (so the test VM exercises the same grant path real users hit, per §6.J — a test environment that can't grant the permission is testing a different thing than production).

Evidence index (line cites)

Fact	File:line
Permission DEFINED, fixed name, `signature`	`api-app/.../AndroidManifest.xml:11-13`
Provider `readPermission`, same fixed name	`api-app/.../AndroidManifest.xml:93`
Provider authority = `${apiKeyAuthority}`	`api-app/.../AndroidManifest.xml:91`
Client `<uses-permission>`, same fixed name	`app/.../AndroidManifest.xml:9`
api-app release applicationId	`api-app/build.gradle.kts:71`
api-app debug `.dev` suffix	`api-app/build.gradle.kts:150`
apiKeyAuthority release placeholder	`api-app/build.gradle.kts:95`
apiKeyAuthority debug placeholder	`api-app/build.gradle.kts:158`
api-app signing reuses :app block	`api-app/build.gradle.kts:14-17, 116-129`
client release applicationId	`app/build.gradle.kts:51`
client debug `.dev` suffix	`app/build.gradle.kts:169`
client API_TARGET_PACKAGE release/debug	`app/build.gradle.kts:116, 173`
client signing block	`app/build.gradle.kts:135-148`